Executable-Rule Ablation on the Seeded GPT-5.5 Subset

SysML2-NL Ablation Study

Abstract

We evaluate three SysML v2 generation sources on the same seeded 10% subset of the agent-generated corpus: the existing pipeline output (Ours), Codex using GPT-5.5, and direct GPT-5.5 through the Vercel AI Gateway. The evaluation uses the repository's text-first executable-rule checker, `script/sysml_executable_rules.py`, over 155 samples selected with seed 0 from `dataset/data/000387--001935`. Ours obtains the highest aggregate executable-rule score, but all systems retain a substantial weakness on signal-like accept-event outputs. The result suggests that the pipeline's structural grounding improves executable-model compliance relative to direct single-shot GPT-5.5 generation, while the remaining error mode is concentrated enough to motivate targeted repair.

1 Experimental Setup

The subset is exactly the `--random_10per` selection used by `batch_vercel_gpt55.py`: 155 samples, drawn with `random.Random(0)` from all eligible generated prompts. For each selected sample ID, the three artifacts compared are:

System	File pattern	Interpretation
Ours	`dataset/data/<id>/<id>.sysml`	Pipeline-generated corpus artifact
Codex+GPT-5.5	`dataset/data/<id>/<id>.codex.sysml`	Codex single-shot generation with GPT-5.5
GPT-5.5	`dataset/data/<id>/<id>.gpt55.sysml`	Direct Vercel AI Gateway GPT-5.5 generation

Each artifact is scored by the local executable-rule checker, `script/sysml_executable_rules.py`, on the five executable-modeling rules shown in Table 2. The rule taxonomy follows the SysML 2 Rule Verification Guide [1]. The checker assigns scores of 1.0 for `pass` and `not_applicable`, 0.5 for `unsupported`, and 0.0 for `fail`. The aggregate score is the mean over all 155 samples and five rules (775 rule evaluations per system).

2 Results

Table 1: Aggregate executable-rule outcomes on the selected 10% subset.

System	Mean score	All-pass samples	Pass	Fail	Unsupported
Ours	0.8968	80 / 155	695	80	0
GPT-5.5	0.8710	57 / 155	675	100	0
Codex+GPT-5.5	0.8600	47 / 155	666	108	1

Ours has the highest mean executable-rule score and the largest number of samples passing all five checks. The direct GPT-5.5 run is slightly stronger than the Codex+GPT-5.5 run in this measurement, but the margin is small.

Table 2: Per-rule pass counts. Each row has 155 evaluations per system.

Rule	Ours	GPT-5.5	Codex+GPT-5.5
`ACCEPTEVENTOUTPUT`	83 pass / 72 fail	57 pass / 98 fail	48 pass / 107 fail
`MESSAGEFLOWNEEDED`	155 pass	155 pass	155 pass
`MESSAGESIGNATURE`	154 pass / 1 fail	155 pass	155 pass
`STMINTEGRITY`	148 pass / 7 fail	153 pass / 2 fail	154 pass / 1 fail
`SUBMACHINESTR`	155 pass	155 pass	154 pass / 1 unsupported

The dominant failure mode is `ACCEPTEVENTOUTPUT`. Ours reduces this failure count from 98–107 in the GPT-5.5 baselines to 72, but it does not eliminate the issue. Conversely, the GPT-5.5 baselines are slightly cleaner on `STMINTEGRITY`, where Ours has seven failures. This indicates that the pipeline improves event-payload modeling more than it improves state-machine call ownership.

Table 3: Paired per-sample score differences. Confidence intervals are descriptive normal approximations over the 155 paired samples.

Comparison	Mean difference	95% CI	Better / equal / worse
Ours − Codex+GPT-5.5	+0.0368	[0.0128, 0.0607]	60 / 66 / 29
Ours − GPT-5.5	+0.0258	[0.0026, 0.0490]	49 / 78 / 28
GPT-5.5 − Codex+GPT-5.5	+0.0110	[-0.0105, 0.0325]	40 / 85 / 30

The paired analysis supports two conclusions. First, Ours has a consistent but modest advantage over both single-shot baselines on executable-rule compliance. Second, the difference between direct GPT-5.5 and Codex+GPT-5.5 is not robust under this metric: most samples are tied, and the confidence interval spans zero.

3 Interpretation

The ablation points to a concentrated compliance gap rather than broad syntactic degradation. Message-flow realization, message signatures, and submachine structure are almost saturated for all three systems. The main remaining issue is whether signal-like `accept` constructs expose a typed output or payload. This is a semantically narrow but executable-critical failure: a model can look structurally complete while still omitting the data interface needed by downstream behavior.

The pipeline output is strongest overall, plausibly because its generation path uses retrieval, synthesis, and syntax-oriented filtering rather than a single completion. However, its lower `STMINTEGRITY` score shows that this advantage is not uniform. A practical next repair pass should therefore be rule-targeted: add a post-generation check that rewrites signal-like `accept` statements with explicit typed payloads, while separately verifying that state-machine entry, exit, effect, and `perform` calls resolve to available action or operation definitions.

4 Limitations

The executable-rule checker is deliberately text-first and approximate; it is useful for bulk comparison but is not a substitute for a full SysML v2 parser or semantic resolver. The subset is fixed by seed 0 and contains only generated corpus prompts, so the results should be interpreted as an ablation on this study slice rather than a claim about all SysML modeling tasks. Finally, the scoring function treats `not_applicable` as pass and `unsupported` as half credit, which is appropriate for screening but can hide differences in model expressiveness.

References

[1] Brandon Soeder. Sysml 2 rule verification guide. https://github.com/bsoeder/sysml2viz/blob/main/analysis/sysml2_rule_verification_guide.md, 2026. Accessed 2026-06-18.